

```

import numpy as np
import scipy.signal as signal
import matplotlib.pyplot as plt

MESSAGE_FREQ1 = 4000.0
MESSAGE_FREQ2 = 3000.0
CARRIER_FREQ1 = 90e6
CARRIER_FREQ2 = 91e6
MOD_INDEX1 = 5.0
MOD_INDEX2 = 6.0
FREQ_DEV1 = MOD_INDEX1 * MESSAGE_FREQ1
FREQ_DEV2 = MOD_INDEX2 * MESSAGE_FREQ2
BANDWIDTH = 100e3
IF_FREQ = 10e6
LO_FREQ = IF_FREQ + CARRIER_FREQ2
SAMPLING_FREQ = 200e6
SIGNAL_DURATION = 0.001
ATTENUATION1 = 3e8 / (CARRIER_FREQ1 * 4 * np.pi * 2000.0)
ATTENUATION2 = 3e8 / (CARRIER_FREQ2 * 4 * np.pi * 3000.0)
AMPLITUDE1 = 5 / ATTENUATION1
AMPLITUDE2 = 5 / ATTENUATION2

def convo_out(m_t, g_t):
    x_t = np.convolve(m_t, g_t, 'same')
    return x_t

def design_sinc_filter(B, fs):
    filter_duration = min(0.1, 5 / B)
    t_start = -filter_duration
    t_stop = filter_duration
    duration = t_stop - t_start
    num_samples = int(fs * duration) + 1
    time_vector = np.linspace(t_start, t_stop, num_samples)
    impulse_response = 2 * B * np.sinc(2 * B * time_vector)
    normalization_factor = np.sum(np.abs(impulse_response))
    if normalization_factor > 0:
        impulse_response /= normalization_factor + 1e-16
    return impulse_response, time_vector

def design_bandpass_filter(B, fc_center, fs):
    g_lp, t_h = design_sinc_filter(B, fs)
    g_bp = g_lp * 2 * np.cos(2 * np.pi * fc_center * t_h)
    return g_bp

def compute_spectrum(m_t, fs):
    M_f = np.fft.fft(m_t)
    M_f_arrange = np.fft.fftshift(M_f)
    freq_axis = np.linspace(-fs / 2, fs / 2, len(M_f))
    return freq_axis, M_f_arrange

def reconstruct_from_spectrum(M_f, fs):
    M_f = np.fft.ifftshift(M_f)
    m_t = np.fft.ifft(M_f)
    t = np.linspace(0, len(m_t) / fs, len(m_t), endpoint=False)
    return t, m_t

def fm_mod(m_t, fc, t, kf, fs):
    a_t = np.cumsum(m_t) / fs
    x_t = np.cos(2 * np.pi * (fc * t + kf * a_t))
    return x_t

def compute_analytic_signal(input_signal):
    return signal.hilbert(input_signal)

def compute_phase(signal_analytic):
    return np.unwrap(np.angle(signal_analytic))

def compute_instantaneous_frequency(phase, dt):
    return np.gradient(phase, dt) / (2 * np.pi)

def fm_demod(m_t, t, fc, fs, fm, kf):

```

```

    dt = t[1] - t[0]
    analytic = compute_analytic_signal(m_t)
    phase = compute_phase(analytic)
    inst_freq = compute_instantaneous_frequency(phase, dt)
    x_t = (inst_freq - fc) / kf
    x_t = x_t - np.mean(x_t)
    lpf_taps = design_sinc_filter(fm, fs)[0]
    x_t = convo_out(x_t, lpf_taps)
    return x_t

def main():
    time = np.arange(0, SIGNAL_DURATION, 1.0 / SAMPLING_FREQ)

    message1 = np.sin(2 * np.pi * MESSAGE_FREQ1 * time)
    message2 = np.cos(2 * np.pi * MESSAGE_FREQ2 * time)

    signal1 = AMPLITUDE1 * fm_mod(
        message1, CARRIER_FREQ1, time, FREQ_DEV1, SAMPLING_FREQ
    )
    signal2 = AMPLITUDE2 * fm_mod(
        message2, CARRIER_FREQ2, time, FREQ_DEV2, SAMPLING_FREQ
    )

    signal1 = ATTENUATION1 * signal1
    signal2 = ATTENUATION2 * signal2

    received_signal = signal1 + signal2

    noise_signal = np.random.randn(len(received_signal)) * np.sqrt(1) + 0
    noisy_signal = received_signal + noise_signal

    bandpass_filter = design_bandpass_filter(BANDWIDTH, CARRIER_FREQ2, SAMPLING_FREQ)
    filtered_signal = convo_out(noisy_signal, bandpass_filter)

    mixer_output = filtered_signal * np.cos(2 * np.pi * LO_FREQ * time)

    if_filter = design_bandpass_filter(BANDWIDTH, IF_FREQ, SAMPLING_FREQ)
    if_signal = convo_out(mixer_output, if_filter)

    demodulated_signal = fm_demod(
        if_signal, time, IF_FREQ, SAMPLING_FREQ, fm=BANDWIDTH, kf=FREQ_DEV2
    )

    plt.figure(figsize=(10, 4))
    zoom_duration = 5e-6
    zoom_samples = int(zoom_duration * SAMPLING_FREQ)
    if zoom_samples < 2:
        zoom_samples = min(len(time), 1000)
    plt.plot(time[:zoom_samples], received_signal[:zoom_samples])
    plt.title("Time Domain Signal (zoom)")
    plt.xlabel("Time (s)")
    plt.ylabel("Amplitude")
    plt.grid(True)
    plt.tight_layout()
    plt.show()

    plt.figure(figsize=(10, 4))
    freq, Rf = compute_spectrum(received_signal, SAMPLING_FREQ)
    plt.plot(freq, np.abs(Rf))
    plt.xlim(85e6, 95e6)
    plt.title("Modulated Frequency Domain (linear magnitude)")
    plt.xlabel("Frequency (Hz)")
    plt.ylabel("Magnitude")
    plt.grid(True)
    plt.tight_layout()
    plt.show()

if __name__ == "__main__":
    main()

```